



**FATİH
SULTAN
MEHMET**
VAKIF UNIVERSITY

SNAKES & LADDERS

Multiplayer Network Game

Project Report

Author: İkbâl Meryem Kaya
Student ID: 2221251013

Code Language: Java

Course: Computer Networks and Design
Instructor: Samet Kaya

1. Introduction

This project implements a two-player, networked version of the classic board game Snakes & Ladders in Java. The goal was to develop a fully functional client–server application that allows two players located on different machines to connect over the internet, wait in a shared lobby, challenge each other, and play a complete game in real time.

The server is deployed on an Amazon Web Services (AWS) EC2 instance, making the game accessible from any machine with a network connection. The client application is a Java Swing desktop program with a graphical interface covering the login screen, lobby, game board, and end screen.

2. System Overview

The application follows a classic client–server architecture. A single server process runs continuously on the AWS EC2 instance and manages all connected clients. Each client runs locally on a player's machine and communicates with the server via TCP sockets. The diagram below summarises the high-level structure:

Component	Technology	Role
Server	Java (TCP ServerSocket)	Accepts connections, manages lobby & game rooms
Client	Java Swing + Socket	UI, game display, user input
Network	AWS EC2 (IP: 51.21.250.119, Port: 1234)	Hosts the server process
Protocol	Custom text-based (pipe-delimited)	Message exchange between client and server

3. Project Structure

The project is divided into two top-level packages:

3.1 Server Package (server.*)

Class	Responsibility
Server.java	Entry point. Listens on port 1234, accepts connections, creates a ClientHandler per socket.
ClientHandler.java	One thread per connected client. Reads incoming messages and dispatches them.
LobbyManager.java	Tracks all online players, their status (AVAILABLE / PENDING / IN_GAME), and manages invite flow.

Class	Responsibility
GameRoom.java	One instance per active game. Handles dice rolls, board logic (snakes & ladders), win detection, and broadcasts state.
GameState.java	Plain data class holding positions, current player, and game-over flag for an active game.

3.2 Client Package (client.*)

Class	Responsibility
GameClient.java	Background thread. Manages the socket connection to the server, sends messages, and routes incoming messages to the correct UI screen.
StartScreen.java	First window the user sees. Collects username and initiates the connection.
LobbyScreen.java	Displays the list of online players with their status; allows sending and responding to game invitations.
GameScreen.java	Renders the 10×10 Snakes & Ladders board, shows dice results, and handles the Roll button.
EndScreen.java	Shown at game end; displays winner information and offers Play Again / Exit options.

4. Communication Protocol

All messages are plain UTF-8 text, delimited by the pipe character (|). Each message is prefixed with a one-byte length field so the receiver knows exactly how many bytes to read. This avoids the need for a delimiter scan and keeps framing simple.

The table below lists all message types:

Direction	Message	Meaning
Client → Server	LOGIN <username>	Authenticate and join the lobby.
Client → Server	INVITE <target>	Send a game invitation to another player.
Client → Server	INVITE_RESPONSE ACCEPT	Accept a pending invitation.
Client → Server	INVITE_RESPONSE REJECT	Reject a pending invitation.

Direction	Message	Meaning
Client → Server	ROLL_REQUEST	Request a dice roll (only valid on player's turn).
Client → Server	DISCONNECT	Graceful disconnection.
Client → Server	GAME_ENDED	Notify server the player has returned to lobby.
Server → Client	LOBBY_UPDATE u1:STATUS ...	Broadcast updated player list to everyone.
Server → Client	INVITE_REQUEST <sender>	Notify the target that they have been invited.
Server → Client	GAME_START <opponent> FIRST/SECOND	Tell each player the game has begun and their turn order.
Server → Client	GAME_STATE dice pos1 pos2 currentPlayer	Send the full board state after every move.
Server → Client	GAME_OVER <winner>	Announce the game winner.
Server → Client	ERROR <message>	Inform the client of an error or disconnection event.

5. Game Logic

5.1 Board

The board is a standard 10×10 grid numbered 1–100. Snakes and ladders are stored in a single `HashMap<Integer,Integer>` on the server (`GameRoom.java`). After every dice roll, if the new position is a key in the map, the player is moved to the corresponding value — upward for ladders, downward for snakes.

5.2 Turn Flow

- The player whose turn it is clicks Roll.
- The client sends `ROLL_REQUEST` to the server.
- The server validates that it is indeed that player's turn (synchronized method).
- A random integer 1–6 is generated and the new position is calculated.
- Snake/ladder check is performed; position is updated in `GameState`.
- `GAME_STATE` is broadcast to both clients.

- If the new position equals 100, GAME_OVER is sent and both players are returned to the lobby.

5.3 Snakes & Ladders Mapping

Type	From	To
Ladder1	4	17
Ladder2	10	29
Ladder3	22	43
Ladder4	45	58
Ladder5	49	72
Ladder6	66	87
Ladder7	78	97
Snake1	36	6
Snake2	56	18
Snake3	68	31
Snake4	81	40
Snake5	84	65

6. Lobby & Invite System

LobbyManager uses synchronized methods throughout to prevent race conditions when multiple clients interact concurrently. Each player can be in one of three states:

- AVAILABLE — player is idle and can be challenged.
- PENDING — player has sent or received an invitation that has not yet been answered.
- IN_GAME — player is currently playing a game.

When a player sends an INVITE, the server immediately sets both parties to PENDING and forwards an INVITE_REQUEST to the target. On acceptance, a new GameRoom is created and both players are set to IN_GAME. On rejection, both are restored to AVAILABLE. After the game ends (or a player disconnects mid-game), both players are returned to AVAILABLE and a LOBBY_UPDATE is broadcast.

7. Deployment on AWS EC2

The server is deployed on an Amazon EC2 instance. The steps taken were:

- Launched an EC2 instance (Amazon Linux / Ubuntu).
- Opened inbound TCP port 1234 in the Security Group.
- Uploaded the compiled server JAR (SnakesAndLadders.jar) to the EC2 instance via SCP.
- Started a screen session: `screen -S server`
- Launched the server with: `java -jar SnakesAndLadders.jar`
- Detached from the screen session using `Ctrl+A` then `D`, so the server keeps running even after closing the SSH terminal.

The client's `SERVER_IP` constant in `GameClient.java` is set to the public IPv4 address of the EC2 instance (51.21.250.119). No additional configuration is needed on the client side.

8. Known Limitations & Future Work

Limitation	Details
Fixed server IP	The server address is hard-coded in <code>GameClient.java</code> . A configuration file or discovery mechanism would improve flexibility.
No reconnection	If a client loses its connection mid-game, it cannot rejoin. The opponent receives an error and is returned to the lobby.
Message framing	The one-byte length prefix limits messages to 255 bytes. Long error strings could be truncated if they exceed this limit.
Username uniqueness	The server does not currently reject duplicate usernames; a second login with the same name could cause confusion in the lobby.
Single room per pair	Two players can only have one active game at a time between them. The architecture supports multiple simultaneous rooms for different player pairs.

9. Conclusion

The project successfully implements a complete, two-player networked Snakes & Ladders game. The server is live on AWS and handles multiple concurrent connections. The client provides a polished Swing-based interface with separate screens for login, lobby, gameplay, and game-end results. All core features — connection management, lobby broadcasting, invitation flow, turn-based dice rolling, snake and ladder movement, win detection, and opponent disconnection handling — are implemented and functional.